Dataset Flexibility: Students may use a different dataset of their own choice if it can be used to solve the given problem statement, satisfies all specified conditions and requirements, and is at least comparable to or more complex than the dataset provided for the project.

Project 1: Causal NLP Reasoning

Title: Neuro Symbolic Causal Inference via Language Model Knowledge Extraction and Bayesian Networks

Project Overview & Abstract

Build a system that extracts candidate cause and effect relations from scientific or news text, then represents and tests them in a Bayesian network. The goal is to separate fluent language understanding from formal causal reasoning and counterfactual queries.

The 'Hybrid' Logic: Text to DAG Transformation

AML baseline: Use dependency patterns, named entity recognition, or a lightweight relation classifier to create candidate causal triples. Construct a directed acyclic graph and fit a Bayesian network with pgmpy.

Identify the gap: Text extraction can propose plausible relations but cannot by itself enforce acyclicity, quantify uncertainty, or answer intervention queries consistently.

DL model: Fine tune a compact transformer such as DistilBERT for causal relation extraction, or use a carefully evaluated instruction model to label candidate pairs. Feed only reviewed predictions into the graph stage.

Fair comparison: Evaluate relation extraction with precision, recall and F1 on a held out text set; evaluate the final graph separately by edge precision and a small set of known intervention queries.

Suggested Pre-reads

Pearl, J. (1988). Probabilistic Reasoning in Intelligent Systems. Devlin et al. (2019). BERT Pre training of Deep Bidirectional Transformers for Language Understanding.

Datasets & Stack

Recommended starter dataset: Start with Corr2Cause for causal text reasoning; use PubMed abstracts only as an optional domain extension. Corr2Cause gives labelled cause effect reasoning examples before moving to open text.

Dataset Link: Corr2Cause - official dataset

Suggested stack: Python, pandas, scikit learn, Hugging Face Transformers, PyTorch, spaCy, NetworkX and pgmpy.

Feature Engineering & Setup Tips

Define a small entity schema first. Deduplicate entity aliases, retain sentence and document identifiers for every edge, reject self loops, and split documents before extraction to prevent text leakage.

Expected Deliverable

A reproducible notebook that extracts relations, visualizes a DAG, compares AML and transformer extraction, and answers a few documented causal queries.

Project 2: Turbulence and Weather Field Prediction

Title: Turbulence Prediction via Physics Informed Neural Networks and ConvLSTMs

Project Overview & Abstract

Forecast short horizon velocity, pressure, or precipitation fields from gridded weather data while using a physics residual to discourage implausible flow predictions. This is a carefully scoped surrogate modelling project, not a full climate model.

The 'Hybrid' Logic: Feature Engineered Forecasting to Physics Constrained Spatiotemporal Learning

AML baseline: Aggregate each grid cell or region into lagged temperature, pressure, humidity and wind statistics; train Ridge Regression, Random Forest or XGBoost to forecast the next value.

Identify the gap: Tabular forecasts lose spatial neighbourhood structure and can predict fields that violate basic smoothness or conservation constraints.

DL model: Train a small ConvLSTM on weather patches. Add a scaled divergence or PDE residual only after a plain ConvLSTM baseline is working.

Fair comparison: Use chronological and geographically separated splits. Report MAE and RMSE by lead time, plus a physics diagnostic such as divergence error.

Suggested Pre-reads

Raissi et al. (2019). Physics Informed Neural Networks. Shi et al. (2015). Convolutional LSTM Network for Precipitation Nowcasting.

Datasets & Stack

Recommended starter dataset: Begin with a small ERA5 reanalysis subset from the Copernicus Climate Data Store, limited to one variable group, region and season. NOAA daily weather data is a simpler tabular baseline.

Dataset Link: ERA5 hourly data on single levels - Copernicus CDS

Suggested stack: Python, xarray, dask, NumPy, scikit learn, PyTorch and matplotlib.

Feature Engineering & Setup Tips

Standardize each meteorological variable using training dates only. Build fixed spatial patches and lag windows, preserve latitude longitude ordering, and use a chronological split with a gap between train and test periods.

Expected Deliverable

A next step field forecast comparison between XGBoost and ConvLSTM, including maps of error and a short physics consistency analysis.

Project 3: Phonetic Guided Speech Generation

Title: Phonetic Guided Audio Generation via Alignment and Latent Diffusion

Project Overview & Abstract

Create a small text to speech experiment that compares explicit phoneme duration alignment with a learned acoustic generator. The practical deliverable is a short, intelligible speech synthesis prototype rather than a production voice system.

The 'Hybrid' Logic: Explicit Phoneme Alignment to Learned Acoustic Generation

AML baseline: Use Montreal Forced Aligner or a conventional HMM aligner to map transcript phonemes to time spans. Train a simple duration predictor as the baseline.

Identify the gap: Alignment alone specifies when a phoneme occurs but does not generate natural timbre, coarticulation, or prosody.

DL model: Fine tune a small pretrained TTS model or train a mel spectrogram decoder conditioned on phonemes and durations. Diffusion is an optional advanced extension after a baseline works.

Fair comparison: Use speaker disjoint evaluation where possible. Report word error rate from a held out ASR model, duration MAE, and a small human intelligibility preference study.

Suggested Pre-reads

McAuliffe et al. (2017). Montreal Forced Aligner. Popov et al. (2021). Grad TTS.

Datasets & Stack

Recommended starter dataset: Start with LJSpeech for a single speaker, English prototype. Use VCTK only if multi speaker variation is a deliberate extension.

Dataset Link: LJ Speech 1.1 - audio and transcripts

Suggested stack: Python, PyTorch, torchaudio, librosa, Montreal Forced Aligner and a pretrained TTS toolkit.

Feature Engineering & Setup Tips

Resample all clips to one rate, normalize transcripts, exclude failed alignments, and keep speaker and utterance identifiers when splitting. Cache mel spectrograms and alignments to make experiments reproducible.

Expected Deliverable

A short synthesis demo comparing duration only alignment with neural acoustic generation and documenting intelligibility failures.

Project 4: Traffic Flow Control

Title: Traffic Flow Prediction and Signal Control via Graph Attention Networks and Flow Models

Project Overview & Abstract

Predict congestion in a road network and test whether graph based learning improves on independent road segment forecasts. Signal control should be simulated only; the project must not claim live traffic control.

The 'Hybrid' Logic: Segment Level Flow Features to Network Aware Graph Learning

AML baseline: Create lagged speed, flow, occupancy, time of day and road segment features; train XGBoost per segment or a multi output Random Forest.

Identify the gap: Independent segment models cannot directly represent how congestion propagates through upstream and downstream links.

DL model: Train a Graph Attention Network or diffusion graph model over the road graph, using METR LA sensor series and a fixed adjacency matrix. Use SUMO only for optional simulation.

Fair comparison: Use chronological splits and report MAE, RMSE and MAPE by forecast horizon. Compare equal history windows and report performance for peak versus off peak periods.

Suggested Pre-reads

Veličković et al. (2018). Graph Attention Networks. Zheng et al. (2019). CityFlow.

Datasets & Stack

Recommended starter dataset: Begin with METR LA traffic speed data, which includes a road graph. PeMS can be used later for a larger California extension.

Dataset Link: METR-LA - traffic series and sensor graph

Suggested stack: Python, pandas, scikit learn, PyTorch, PyTorch Geometric, NetworkX and optional SUMO.

Feature Engineering & Setup Tips

Construct the adjacency matrix from the supplied graph, fit scaling only on training timestamps, include cyclic time features, and never randomly split sequences. Mask sensor outages instead of treating them as zero traffic.

Expected Deliverable

A forecast comparison with congestion maps, horizon wise metrics, and an optional simulated signal policy experiment.

Project 5: Inventory Demand Forecasting

Title: Inventory Demand Forecasting via Kalman Filtering and Global DeepAR Models

Project Overview & Abstract

Forecast demand for many retail series while comparing local state tracking with a global probabilistic neural forecast. The project emphasizes fair handling of stockouts, cold start items and forecast uncertainty.

The 'Hybrid' Logic: Local State Estimation to Global Sequence Learning

AML baseline: Build seasonal naive and gradient boosted lag feature baselines. Use a Kalman filter per SKU to estimate local demand state and uncertainty.

Identify the gap: Local filters adapt quickly but have limited ability to share seasonal patterns across sparse product histories.

DL model: Train a DeepAR or LSTM global model with static item identifiers and dynamic calendar features. Use predicted quantiles, not only point forecasts.

Fair comparison: Backtest on rolling origins. Report MAE, RMSE, WAPE and pinball loss for forecast intervals; compare at the same horizons.

Suggested Pre-reads

Kalman (1960). A New Approach to Linear Filtering and Prediction Problems. Salinas et al. (2020). DeepAR.

Datasets & Stack

Recommended starter dataset: Start with the M5 Forecasting Accuracy data for a limited set of product series. The Walmart recruiting data is a good secondary retail benchmark.

Dataset Link: M5 Forecasting - Accuracy: competition data

Suggested stack: Python, pandas, statsmodels or filterpy, scikit learn, PyTorch Forecasting or GluonTS.

Feature Engineering & Setup Tips

Use a complete date index per SKU, mark rather than impute stockout periods, create lag and rolling features without peeking ahead, encode calendars, and split by forecast origin.

Expected Deliverable

A rolling forecast report that compares a local filter, a tabular AML baseline and a global DL model with uncertainty intervals.

Project 6: Urban Microclimate Modelling

Title: Microclimate Modelling via Neural Surrogates and Computational Fluid Dynamics

Project Overview & Abstract

Learn a fast surrogate for wind or heat fields around simplified urban layouts, then compare it with a small collection of CFD simulations. The study is about surrogate accuracy and uncertainty, not safety critical design approval.

The 'Hybrid' Logic: Geometric Summary Features to Spatial Surrogate Fields

AML baseline: Describe each layout with building density, height statistics, street orientation and weather inputs; predict area averaged wind speed or heat using Random Forest or XGBoost.

Identify the gap: Layout summaries cannot represent local wake patterns, street canyon effects, or detailed spatial variation.

DL model: Encode a rasterized building layout and conditions as channels; train a compact U Net or conditional CNN to predict a velocity or temperature field.

Fair comparison: Hold out entire layouts rather than random pixels. Report field MAE, normalized RMSE, and error maps for unseen geometries.

Suggested Pre-reads

Goodfellow et al. (2014). Generative Adversarial Nets. Farimani et al. (2017). Deep Learning for Wall Bounded Turbulent Flows.

Datasets & Stack

Recommended starter dataset: Begin with a self generated OpenFOAM sweep over simple two dimensional building blocks. Use a small curated set of layouts before considering Cityscapes derived geometry.

Dataset Link: UrbanTALES - idealized building arrays (CFD alternative)

Suggested stack: OpenFOAM or a documented CFD surrogate dataset, Python, NumPy, PyTorch, xarray and matplotlib.

Feature Engineering & Setup Tips

Store geometry, boundary conditions and solver settings with every field. Normalize velocities by inlet speed, align raster coordinate systems, and split by layout family to test generalization.

Expected Deliverable

A surrogate comparison with field visualizations, held out layout errors and a clear statement of simulation limits.

Project 7: Remaining Useful Life Prediction for Industrial Engines

Title: Predictive Maintenance via Sensor Features and Sequence Models

Project Overview & Abstract

Estimate the remaining operating cycles before an engine fails from multivariate sensor readings. The key comparison is between engineered degradation features and a model that learns directly from sensor sequences.

The 'Hybrid' Logic: Degradation Feature Engineering to Sequence Based Remaining Life Prediction

AML baseline: Create cycle, rolling mean, rolling slope, operating regime and sensor health features; train Random Forest, XGBoost or Elastic Net regression.

Identify the gap: Fixed rolling features may miss long degradation trajectories and interactions whose timing differs across engines.

DL model: Train an LSTM, temporal CNN or Transformer encoder on fixed length sensor windows to predict RUL.

Fair comparison: Split by engine ID, not individual rows. Report RMSE and MAE on RUL, plus error by early versus late degradation stage.

Suggested Pre-reads

Saxena and Goebel (2008). Turbofan Engine Degradation Simulation Data Set. Hochreiter and Schmidhuber (1997). Long Short Term Memory.

Datasets & Stack

Recommended starter dataset: NASA C MAPSS Turbofan Engine Degradation Dataset, starting with subset FD001.

Dataset Link: NASA C-MAPSS Turbofan Engine Degradation

Suggested stack: Python, pandas, scikit learn, PyTorch or TensorFlow, matplotlib and optional SHAP.

Feature Engineering & Setup Tips

Compute RUL only from the final cycle of each training engine, cap the target only if stated, normalize sensors by training operating regime, remove flat sensors after inspection, and make windows within engine boundaries.

Expected Deliverable

A predictive maintenance notebook comparing engineered feature regression and sequence learning with engine level test results.

Project 8: Molecular Toxicity Prediction

Title: Molecular Toxicity Prediction via Fingerprints and Molecular Graph Networks

Project Overview & Abstract

Predict toxicity assay activity from molecular structures and investigate whether graph learned representations outperform engineered chemical fingerprints.

The 'Hybrid' Logic: Chemical Fingerprints to Message Passing Molecular Representations

AML baseline: Convert SMILES strings to Morgan fingerprints and basic descriptors; train Logistic Regression, Random Forest and XGBoost for each endpoint.

Identify the gap: Fingerprints encode fixed substructures and may not capture the task specific interactions needed across toxicity endpoints.

DL model: Train a graph convolution or message passing network using atoms as nodes and bonds as edges, with masked loss for missing labels.

Fair comparison: Use scaffold splits to test unseen chemical families. Report endpoint wise ROC AUC and PR AUC, excluding missing labels from each endpoint metric.

Suggested Pre-reads

Wu et al. (2018). MoleculeNet. Gilmer et al. (2017). Neural Message Passing for Quantum Chemistry.

Datasets & Stack

Recommended starter dataset: Tox21 from MoleculeNet or DeepChem; it contains roughly 7,800 compounds and 12 toxicity endpoints.

Dataset Link: Tox21 - DeepChem data (CSV.GZ)

Suggested stack: Python, RDKit, scikit learn, DeepChem or PyTorch Geometric, PyTorch and pandas.

Feature Engineering & Setup Tips

Canonicalize SMILES, deduplicate compounds before splitting, preserve missing labels with a mask, compute class weights endpoint by endpoint, and use scaffold rather than random splitting.

Expected Deliverable

An endpoint wise comparison of fingerprint AML and graph DL models, including a scaffold generalization analysis.

Project 9: Indoor Location Prediction Using Wi Fi Signals

Title: Indoor Positioning via Signal Strength Features and Deep Multitask Learning

Project Overview & Abstract

Predict a device building, floor and position from sparse Wi Fi access point measurements. The project naturally combines classification and coordinate regression.

The 'Hybrid' Logic: Sparse Signal Features to Multitask Location Learning

AML baseline: Impute absent access points consistently, add signal count and strongest signal features, then train Random Forest or XGBoost classifiers for building and floor plus a regressor for coordinates.

Identify the gap: Separate tabular models may not share useful information among building, floor and coordinate targets or represent complex interactions among hundreds of access points.

DL model: Train a shared MLP trunk with classification heads for building and floor and a regression head for coordinates.

Fair comparison: Use the dataset supplied train test partition or a split by user and collection trajectory. Report building and floor accuracy, coordinate MAE, and a hierarchical error breakdown.

Suggested Pre-reads

Torres Sospedra et al. (2014). UJIIndoorLoc. Caruana (1997). Multitask Learning.

Datasets & Stack

Recommended starter dataset: UJIIndoorLoc from the UCI Machine Learning Repository.

Dataset Link: [UJIIndoorLoc - UCI Machine Learning Repository]

Suggested stack: Python, pandas, scikit learn, PyTorch or TensorFlow, NumPy and matplotlib.

Feature Engineering & Setup Tips

Treat the dataset missing signal convention consistently, do not normalize test data separately, add missingness indicators, scale observed RSSI values, and retain user and phone identifiers for robustness analysis.

Expected Deliverable

A multitask positioning notebook with AML and DL comparison, floor confusion matrices and coordinate error maps.

Project 10: Higgs Boson Signal Detection

Title: Particle Event Classification via Expert Features and Neural Networks

Project Overview & Abstract

Classify simulated particle collision events as signal or background, comparing models built on expert features with neural networks trained on the same numerical inputs.

The 'Hybrid' Logic: Expert Derived Tabular Features to Learned Numerical Interactions

AML baseline: Train Logistic Regression and XGBoost on standardized low level features, then repeat with expert derived features for an interpretable baseline.

Identify the gap: Tree ensembles and linear models can be strong, but may be less efficient at learning complex smooth interactions from raw measurements at scale.

DL model: Train a compact MLP with batch normalization and dropout; use data loading that supports a stratified subset before attempting the full dataset.

Fair comparison: Use one held out test set and compare ROC AUC, PR AUC, calibration and training time. Report results for a fixed subset budget before scaling.

Suggested Pre-reads

Baldi et al. (2014). Searching for Exotic Particles in High Energy Physics. Glorot and Bengio (2010). Understanding the Difficulty of Training Deep Feedforward Networks.

Datasets & Stack

Recommended starter dataset: HIGGS dataset from UCI. Begin with a documented stratified subset of 250,000 to 1,000,000 events rather than the full 11 million rows.

Dataset Link: HIGGS - UCI Machine Learning Repository

Suggested stack: Python, pandas or Polars, scikit learn, PyTorch, matplotlib and optional SHAP.

Feature Engineering & Setup Tips

Read in chunks, remove the label before scaling, fit transformations on training data only, make the subset seed explicit, and compare models at the same data budget.

Expected Deliverable

A scalable event classification benchmark showing accuracy, calibration and compute trade offs.

Project 11: Automatic Clothing Category Recognition

Title: Inventory Image Tagging via Handcrafted Features and a Small CNN

Project Overview & Abstract

Classify basic clothing images for a small catalogue and examine whether a CNN resolves visually similar categories better than fixed image descriptors.

The 'Hybrid' Logic: Handcrafted Visual Features to Hierarchical Visual Features

AML baseline: Normalize the 28 by 28 images; compare flattened pixels and HOG descriptors with Logistic Regression, k nearest neighbors, Random Forest and SVM.

Identify the gap: Flattened pixels discard neighbourhood structure, while HOG fixes the representation before the model sees category specific examples.

DL model: Train a two or three block CNN with pooling, dropout and early stopping.

Fair comparison: Use the official train test split with a validation set drawn only from training data. Report accuracy, macro F1, per class recall, parameter count and inference time.

Suggested Pre-reads

Xiao et al. (2017). Fashion MNIST. LeCun et al. (1998). Gradient Based Learning Applied to Document Recognition.

Datasets & Stack

Recommended starter dataset: Fashion MNIST, available through torchvision or TensorFlow datasets.

Dataset Link: [Fashion-MNIST - official Zalando dataset]

Suggested stack: Python, NumPy, scikit image, scikit learn, PyTorch or TensorFlow and matplotlib.

Feature Engineering & Setup Tips

Scale pixels to zero one, keep the class labels intact, compute HOG only after fixing image orientation, and inspect confusion between shirts, T shirts and pullovers.

Expected Deliverable

A classifier notebook with AML versus CNN confusion examples and a concise error analysis.

Project 12: Smart Waste Segregation Assistant

Title: Everyday Waste Sorting via Manual Image Descriptors and Transfer Learning

Project Overview & Abstract

Build an educational photo classifier that suggests a material class from an image, while clearly documenting that local recycling rules and contamination determine real disposal decisions.

The 'Hybrid' Logic: Fixed Image Descriptors to Reusable Learned Representations

AML baseline: Extract color histograms, HOG and Local Binary Pattern features; train SVM and Random Forest classifiers.

Identify the gap: Manual descriptors can work in controlled photographs but are brittle under lighting, viewpoint, background and object deformation changes.

DL model: Use MobileNetV2 or EfficientNet as a frozen feature extractor, then train a small classification head with modest augmentation.

Fair comparison: Use a stratified split after duplicate checks. Report macro F1, class recall, confidence calibration and qualitative failure cases.

Suggested Pre-reads

Thung and Yang (2016). Classification of Trash for Recyclability Status. Sandler et al. (2018). MobileNetV2.

Datasets & Stack

Recommended starter dataset: TrashNet, beginning with its six material categories. Treat labels as visual material classes, not universal disposal instructions.

Dataset Link: TrashNet - official six-class image dataset

Suggested stack: Python, OpenCV or scikit image, scikit learn, PyTorch or TensorFlow and optional Gradio.

Feature Engineering & Setup Tips

Deduplicate near identical images, crop or letterbox consistently, use conservative augmentation, balance class sampling, and add a low confidence abstain response in any demo.

Expected Deliverable

A photo upload prototype with AML and transfer learning comparison, confidence scores and a visible limitations note.

Project 13: Urban Emergency Sound Recognition

Title: City Sound Triage via Aggregated MFCC Features and a Spectrogram CNN

Project Overview & Abstract

Classify urban environmental sounds such as sirens and car horns using published audio folds. The output is a research triage prototype, not a public safety alarm system.

The 'Hybrid' Logic: Aggregated Audio Statistics to Time Frequency Pattern Learning

AML baseline: Extract MFCC means and standard deviations, zero crossing rate and spectral contrast; compare SVM, Random Forest and Logistic Regression.

Identify the gap: Aggregation erases when a sound changes over time and can confuse events with similar average spectra.

DL model: Convert clips to fixed size log mel spectrograms and train a small 2D CNN.

Fair comparison: Respect the official UrbanSound8K folds. Report mean fold accuracy, macro F1, class recall and fold wise variation.

Suggested Pre-reads

Salamon et al. (2014). A Dataset and Taxonomy for Urban Sound Research. Piczak (2015). Environmental Sound Classification with CNNs.

Datasets & Stack

Recommended starter dataset: UrbanSound8K, using the official ten folds for evaluation.

Dataset Link: [UrbanSound8K - official audio and ten folds]

Suggested stack: Python, librosa, NumPy, scikit learn, PyTorch or TensorFlow and matplotlib.

Feature Engineering & Setup Tips

Resample audio uniformly, trim or pad to a fixed duration, compute features with the same FFT settings, avoid mixing clips from one fold across splits, and inspect class imbalance.

Expected Deliverable

An audio classification report with fold wise AML versus CNN metrics and sound examples for major false alarms.

Project 14: Pneumonia Screening from Chest X Rays

Title: Pneumonia Screening Research Prototype via HOG SVM and Transfer Learned CNNs

Project Overview & Abstract

Compare handcrafted texture and edge features with transfer learned image representations for classifying provided chest X rays as normal or pneumonia. This is an educational research exercise, not a diagnostic tool.

The 'Hybrid' Logic: Kernel Mapped Handcrafted Features to Learned Hierarchical Features

AML baseline: Resize and normalize images, extract HOG and optional LBP descriptors, then tune linear and RBF kernel SVMs.

Identify the gap: A kernel can reshape fixed descriptors but cannot recover lung opacity texture or contextual image evidence discarded during handcrafted feature extraction.

DL model: Fine tune a pretrained ResNet18 or MobileNetV2, beginning with frozen features and then unfreezing only late layers if validation supports it.

Fair comparison: Use patient level separation if identifiers are available; otherwise retain the dataset partition and disclose the limitation. Report pneumonia recall, specificity, ROC AUC and calibration.

Suggested Pre-reads

Boser et al. (1992). A Training Algorithm for Optimal Margin Classifiers. Rajpurkar et al. (2017). CheXNet.

Datasets & Stack

Recommended starter dataset: Chest X Ray Images Pneumonia dataset on Kaggle, with its published normal and pneumonia folders.

Dataset Link: Chest X-Ray Images (Pneumonia) - Kaggle

Suggested stack: Python, OpenCV, scikit image, scikit learn, PyTorch or TensorFlow and matplotlib.

Feature Engineering & Setup Tips

Inspect image duplicates and view artifacts, standardize grayscale intensity, use modest augmentation only on training data, record class prevalence, and never infer clinical performance from this dataset alone.

Expected Deliverable

A well caveated model comparison with class sensitive metrics, saliency examples and a not for clinical use statement.

Project 15: Heart Disease Risk Modelling

Title: Heart Disease Risk Modelling via Bayesian Networks and Feed Forward Neural Networks

Project Overview & Abstract

Compare an explicit domain structured probabilistic model with a neural network that learns nonlinear risk factor interactions from patient records. It is a prediction exercise, not clinical decision support.

The 'Hybrid' Logic: Hand Specified Independence Structure to Learned Feature Interactions

AML baseline: Discretize selected continuous variables, build a small domain informed Bayesian network with pgmpy, and compare it with Naive Bayes.

Identify the gap: Fixed graph structure and coarse discretization can miss continuous and nonlinear interactions among age, heart rate, blood pressure and symptoms.

DL model: Train a regularized MLP on scaled numeric features and one hot encoded categorical variables.

Fair comparison: Use stratified cross validation with patient rows kept independent. Report ROC AUC, PR AUC, sensitivity, specificity and calibration.

Suggested Pre-reads

Pearl (1988). Probabilistic Reasoning in Intelligent Systems. UCI Heart Disease Dataset documentation.

Datasets & Stack

Recommended starter dataset: UCI Heart Disease dataset, beginning with the Cleveland processed subset.

Dataset Link: Heart Disease - UCI (processed Cleveland subset)

Suggested stack: Python, pandas, scikit learn, pgmpy, PyTorch or TensorFlow and SHAP.

Feature Engineering & Setup Tips

Define the target version explicitly, encode missing values and categorical features from training data only, document all discretization cut points, and avoid interpreting predicted risk as a medical recommendation.

Expected Deliverable

An interpretable probability comparison between Bayesian network and MLP, with calibration and a transparent limitations section.

Project 16: Part of Speech Tagging

Title: Part of Speech Tagging via Hidden Markov Models and Bidirectional LSTMs

Project Overview & Abstract

Assign grammatical tags to words in sentences while comparing a generative sequence model decoded by Viterbi with a neural sequence tagger that uses context on both sides.

The 'Hybrid' Logic: Markov Emission Assumptions to Contextual Sequence Representation Learning

AML baseline: Estimate HMM transition and emission probabilities with smoothing and decode using the Viterbi algorithm.

Identify the gap: An HMM assumes a limited Markov context and struggles with unknown words and long range syntactic dependencies.

DL model: Train a BiLSTM tagger with word and character or subword embeddings; mask padding tokens in the loss.

Fair comparison: Use the standard train development test split. Report token accuracy, macro F1, unknown word accuracy and a small error analysis by tag.

Suggested Pre-reads

Rabiner (1989). A Tutorial on Hidden Markov Models. Huang et al. (2015). Bidirectional LSTM CRF Models for Sequence Tagging.

Datasets & Stack

Recommended starter dataset: Universal Dependencies English EWT treebank, using its published train development test split.

Dataset Link: Universal Dependencies English EWT - official treebank

Suggested stack: Python, conllu, NumPy, PyTorch, scikit learn and matplotlib.

Feature Engineering & Setup Tips

Keep sentence boundaries, build vocabulary from train only with UNK handling, add start and end states to Viterbi, pad and mask batches correctly, and preserve the official tag mapping.

Expected Deliverable

A sequence tagging benchmark with HMM and BiLSTM results, unknown word analysis and decoded sentence examples.

Project 17: Next Day Wildfire Spread Prediction

Title: Next Day Wildfire Spread Prediction via Tabular AML and Spatial Deep Learning

Project Overview & Abstract

Predict next day burn risk from remote sensing, weather and terrain variables while comparing engineered tabular features with models that retain local spatial context.

The 'Hybrid' Logic: Engineered Environmental Features to Spatial Neighbourhood Learning

AML baseline: Engineer lagged burn activity, local vegetation statistics, weather summaries, drought indicators, elevation and neighbourhood aggregates; train XGBoost or LightGBM.

Identify the gap: Tabular summaries may miss fire front geometry and spatial interactions around boundaries, especially during rapidly changing events.

DL model: Train a compact CNN or U Net on multichannel spatial patches. Add ConvLSTM only if the data genuinely provides multiday sequences.

Fair comparison: Separate events spatially and temporally. Report PR AUC, recall at fixed false positive rates, F1 and ROC AUC; accuracy alone is misleading for rare burns.

Suggested Pre-reads

Huot et al. (2022). Next Day Wildfire Spread. Ronneberger et al. (2015). U Net.

Datasets & Stack

Recommended starter dataset: Next Day Wildfire Spread dataset from Kaggle, starting with a manageable region and time range.

Dataset Link: Next Day Wildfire Spread - Kaggle

Suggested stack: Python, xarray or rasterio, geopandas, scikit learn, PyTorch and matplotlib.

Feature Engineering & Setup Tips

Define the prediction timestamp exactly, crop patches without crossing split boundaries, retain coordinate reference metadata, apply class weighting only on training data, and evaluate by fire event.

Expected Deliverable

A spatially fair wildfire risk comparison with probability maps, PR curves and visual checks for physically plausible patterns.

Project 18: Water Quality Prediction

Title: Water Quality Prediction via Gradient Boosting and Deep Neural Networks

Project Overview & Abstract

Predict a documented water quality class or index from physical and chemical measurements. The team should choose one target definition and avoid mixing incompatible water quality standards.

The 'Hybrid' Logic: Interpretable Tabular Relationships to Learned Nonlinear Chemical Interactions

AML baseline: Use XGBoost or Random Forest on pH, dissolved oxygen, turbidity, temperature, conductivity and dissolved solids.

Identify the gap: Tree models are strong on tables but may not capture smooth combinations or calibrated uncertainty as efficiently when many sensors interact.

DL model: Train a small regularized MLP after a strong AML baseline; use a regression head for an index and a classification head only for a defined class scheme.

Fair comparison: Use a site or time separated split when metadata permits. Report MAE, RMSE and R squared for a continuous target or macro F1 and calibration for class labels.

Suggested Pre-reads

Breiman (2001). Random Forests. Goodfellow et al. (2016). Deep Learning, chapters on feed forward networks.

Datasets & Stack

Recommended starter dataset: Start with the UCI Water Quality data that matches the selected target, or a documented national monitoring subset with a clear water quality index definition.

Dataset Link: Surface Water Quality Monitoring - national data / CCME WQI

Suggested stack: Python, pandas, scikit learn, XGBoost, PyTorch or TensorFlow and SHAP.

Feature Engineering & Setup Tips

Check unit consistency and detection limits, impute within the training split, flag impossible values, scale inputs for the MLP, and do not derive a target index from the same components without acknowledging leakage.

Expected Deliverable

A water quality prediction report comparing XGBoost and MLP with target definition, missing data treatment and feature importance.

Project 19: Air Quality Prediction

Title: Air Quality Forecasting via Gradient Boosting and LSTM Models

Project Overview & Abstract

Forecast short horizon PM2.5 concentration from historical pollutant and weather series, then optionally map the forecast to an AQI category under one clearly stated standard.

The 'Hybrid' Logic: Lagged Environmental Features to Temporal Sequence Forecasting

AML baseline: Create lags, rolling means, calendar features and weather covariates; train XGBoost regression.

Identify the gap: Manually chosen windows may not represent longer temporal dependencies or nonlinear pollutant weather interactions.

DL model: Train an LSTM or temporal CNN on fixed historical windows, producing a point forecast before attempting a hybrid ensemble.

Fair comparison: Use rolling chronological backtesting. Report MAE, RMSE and R squared at a fixed horizon; compare against persistence and seasonal naive baselines.

Suggested Pre-reads

Hyndman and Athanasopoulos. Forecasting Principles and Practice. Hochreiter and Schmidhuber (1997). Long Short Term Memory.

Datasets & Stack

Recommended starter dataset: Begin with the UCI Beijing PM2.5 dataset or a documented OpenAQ city time series with weather variables.

Dataset Link: Beijing PM2.5 - UCI hourly time series

Suggested stack: Python, pandas, scikit learn, XGBoost, PyTorch or TensorFlow, matplotlib and optional Streamlit.

Feature Engineering & Setup Tips

Resample to a regular interval, preserve missingness flags, build lag features after sorting by time, fit scaling on training dates only, and state the forecast horizon and AQI standard.

Expected Deliverable

A PM2.5 forecasting comparison with chronological backtests, baseline errors and an optional AQI visualization.

Project 20: Appliance Energy Forecasting

Title: Appliance Energy Forecasting via Tabular AML and Sequence Models

Project Overview & Abstract

Forecast short term appliance energy demand from indoor sensors, weather and time of day while assessing whether a sequence model adds value beyond careful feature engineering.

The 'Hybrid' Logic: Lag and Calendar Features to Sensor Sequence Learning

AML baseline: Create lagged energy, rolling statistics, time of day, day of week, temperature and humidity features; train XGBoost or Random Forest.

Identify the gap: A fixed feature window may not capture variable temporal dependencies and interactions across multiple sensors.

DL model: Train an LSTM or temporal CNN on multivariate ten minute windows.

Fair comparison: Split chronologically and report MAE, RMSE and R squared by forecast horizon. Include persistence as a baseline.

Suggested Pre-reads

Candanedo et al. (2017). Appliances Energy Prediction dataset. Bai et al. (2018). Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling.

Datasets & Stack

Recommended starter dataset: UCI Appliances Energy Prediction dataset with 19,735 ten minute observations.

Dataset Link: Appliances Energy Prediction - UCI

Suggested stack: Python, pandas, scikit learn, XGBoost, PyTorch or TensorFlow and SHAP.

Feature Engineering & Setup Tips

Sort timestamps and check cadence, use cyclical clock features, avoid rolling windows that include future values, keep weather measurements aligned to target time, and scale inputs for the DL model.

Expected Deliverable

A short horizon energy forecast report comparing a feature engineered AML baseline with a sequence model.

Project 21: Flight Delay Prediction and Rescheduling

Title: Flight Delay Prediction via Gradient Boosting and Temporal Deep Learning

Project Overview & Abstract

Predict whether a flight will arrive substantially delayed from schedule, carrier, route and historical information, then explore an offline rescheduling recommendation rule. The project must not use post departure information when making pre departure predictions.

The 'Hybrid' Logic: Operational Tabular Features to Learned Temporal and Route Patterns

AML baseline: Create schedule, route, carrier, airport congestion, calendar and historical delay features; train Logistic Regression and XGBoost classifiers.

Identify the gap: Manual route and congestion summaries may miss complex interactions between time, airport, carrier and recent delay dynamics.

DL model: Train a compact MLP with learned embeddings for airline, origin and destination. A temporal model is optional only if flight history is correctly ordered.

Fair comparison: Split by date. Report PR AUC, ROC AUC, recall at a selected operational precision threshold and calibration. Compare only features available at the chosen decision time.

Suggested Pre-reads

US Bureau of Transportation Statistics Airline On Time Performance documentation. Guo et al. (2017). On Calibration of Modern Neural Networks.

Datasets & Stack

Recommended starter dataset: Start with the Kaggle Flights dataset cited in the source document, which contains 2013 US flights. BTS on time performance data is a larger official extension.

Dataset Link: Flights - Kaggle (2013 flight records)

Suggested stack: Python, pandas or Polars, scikit learn, XGBoost, PyTorch and matplotlib.

Feature Engineering & Setup Tips

Choose the prediction timestamp first, remove fields only known after departure, derive historical airport features using past dates only, encode cyclic time, and use a chronological validation split.

Expected Deliverable

A delay risk model with calibrated probabilities and an offline what if rescheduling analysis based on explicit assumptions.

Project 22: Wheat Fusarium Head Blight Severity Estimation

Title: Wheat Disease Severity Estimation via Image Features and Transfer Learned CNNs

Project Overview & Abstract

Estimate Fusarium head blight presence or severity from wheat head images, comparing engineered color texture features with transfer learned visual representations.

The 'Hybrid' Logic: Color and Texture Features to Learned Disease Pattern Recognition

AML baseline: Segment or crop the wheat head, extract color indices, hue histograms, HOG and texture descriptors, then train SVM or XGBoost for class or severity prediction.

Identify the gap: Fixed visual descriptors are sensitive to illumination, growth stage and viewpoint and may miss localized infection patterns.

DL model: Fine tune a lightweight CNN for classification or ordinal severity bins. Use a segmentation model only if masks and adequate labels are available.

Fair comparison: Split by field, date or plant group rather than random images. Report macro F1 for severity classes or MAE for continuous severity, plus per condition errors.

Suggested Pre-reads

Jin et al. (2018). Classifying Wheat Hyperspectral Pixels of Healthy Heads and Fusarium Head Blight Disease. Sandler et al. (2018). MobileNetV2.

Datasets & Stack

Recommended starter dataset: Start with the Dryad dataset of phenotype images of wheat heads infected with Fusarium graminearum, including severity measurements. The OFHB dataset is a useful image classification extension.

Dataset Link: Wheat head phenotype images and severity data - Dryad

Suggested stack: Python, OpenCV, scikit image, scikit learn, PyTorch or TensorFlow and matplotlib.

Feature Engineering & Setup Tips

Inspect label definitions and rater agreement, crop backgrounds consistently, standardize color conservatively, preserve field and date metadata, and test whether predictions transfer across lighting conditions.

Expected Deliverable

A disease severity prototype with image feature versus CNN comparison, condition based error analysis and clear agricultural limitations.

Project 23: Object Segmentation

Title: Object Segmentation via Superpixels and Transfer Learned U Net Models

Project Overview & Abstract

Segment foreground objects in images and compare an interpretable classical segmentation pipeline with a neural model that predicts a pixel wise mask.

The 'Hybrid' Logic: Color Texture Region Features to Learned Pixel Wise Representations

AML baseline: Use SLIC superpixels or GrabCut, then classify regions with color, texture and location features using Random Forest or SVM.

Identify the gap: Handcrafted region features and thresholding struggle when object boundaries, texture and illumination vary across images.

DL model: Fine tune a compact U Net with a pretrained encoder using binary masks and a combined Dice plus binary cross entropy loss.

Fair comparison: Split by original image before any crop or augmentation. Report intersection over union, Dice score and boundary examples on the same test images.

Suggested Pre-reads

Achanta et al. (2012). SLIC Superpixels. Ronneberger et al. (2015). U Net.

Datasets & Stack

Recommended starter dataset: Start with the amaye15 object segmentation dataset on Hugging Face for a small prototype. Because masks are model generated, use them only for a learning exercise and inspect label quality; move to a manually annotated dataset for a stronger study.

Dataset Link: amaye15/object-segmentation - images and masks

Suggested stack: Python, OpenCV, scikit image, scikit learn, PyTorch, segmentation models pytorch and matplotlib.

Feature Engineering & Setup Tips

Verify that every image mask pair has the same dimensions, convert masks to one binary convention, inspect overlays before training, use image level splits, and record the mask provenance.

Expected Deliverable

A segmentation notebook with superpixel AML and U Net masks, quantitative overlap metrics and visual failure cases.

Project 24: RF Power Amplifier Optimization

Title: RF Power Amplifier Optimization via Surrogate Regression and Neural Networks

Project Overview & Abstract

Learn a surrogate model that predicts gain, power added efficiency, output power and distortion from RF power amplifier design variables, then search for feasible high performance configurations.

The 'Hybrid' Logic: Interpretable Design Variable Surrogates to Neural Multi Output Response Modelling

AML baseline: Train Random Forest, XGBoost and Gaussian Process regressors on bias, input power, frequency, transistor and matching network variables.

Identify the gap: Tree models can be strong on small simulation sweeps but may produce discontinuous response surfaces and weak extrapolation across coupled design settings.

DL model: Train a regularized multi output MLP to predict all response metrics jointly, then optimize a normalized multi objective score.

Fair comparison: Hold out design configurations, not rows from repeated frequency traces. Report MAE or RMSE per target, Pareto frontier agreement and feasibility constraint violations.

Suggested Pre-reads

Forrester et al. (2008). Engineering Design via Surrogate Modelling. Goodfellow et al. (2016). Deep Learning, feed forward networks.

Datasets & Stack

Recommended starter dataset: Start with the PA and LNA survey data linked in the source document if it includes design to response records. Otherwise generate a small, documented parameter sweep in ADS, AWR or QUCS and export a CSV; do not mix unrelated simulation settings.

Dataset Link: AICircuit PA design sweep (distortion needs added simulations)

Suggested stack: Python, pandas, scikit learn, XGBoost, PyTorch, Optuna and matplotlib.

Feature Engineering & Setup Tips

Keep units consistent, record simulator and device settings, model frequency as an input or split it deliberately, normalize targets for the MLP, and optimize only within the observed design bounds.

Expected Deliverable

A surrogate accuracy comparison and a constrained multi objective design search with a documented Pareto trade off.

Project 25: Microstrip Antenna Parameter Optimization

Title: Microstrip Antenna Optimization via Ensemble Regressors and Neural Surrogates

Project Overview & Abstract

Predict antenna performance from geometry and substrate variables, then propose designs that balance resonance, S11, gain, bandwidth and efficiency under explicit constraints.

The 'Hybrid' Logic: Geometry Feature Regression to Neural Multi Target Electromagnetic Surrogates

AML baseline: Train Random Forest, XGBoost and Gradient Boosting models on geometric and material variables to predict resonant frequency, S11, gain and bandwidth.

Identify the gap: Independent tabular regressors may miss coupled multi target electromagnetic responses and yield inconsistent design recommendations.

DL model: Train a multi task MLP with shared hidden layers and separate output heads for each antenna response.

Fair comparison: Hold out full geometry configurations. Report MAE or RMSE by target, constraint satisfaction rate and error near the desired operating band.

Suggested Pre-reads

Balanis. Antenna Theory. Forrester et al. (2008). Engineering Design via Surrogate Modelling.

Datasets & Stack

Recommended starter dataset: Antenna Parameters Dataset on Mendeley Data, DOI 10.17632/3gxr2vvd9n.2, with simulation sweep records for a microstrip antenna.

Dataset Link: [Antenna Parameters Dataset - Mendeley Data, version 2]

Suggested stack: Python, pandas, scikit learn, XGBoost, PyTorch, Optuna and matplotlib.

Feature Engineering & Setup Tips

Use one unit system, verify which frequency rows correspond to each geometry, group split by configuration, scale targets for the neural model, and reject proposed designs outside the observed variable ranges.

Expected Deliverable

A multi target surrogate benchmark and constrained antenna design search with transparent trade offs.

Project 26: Card Fraud Triage

Title: Card Fraud Triage via Gaussian Mixture Density Scoring and Autoencoder Anomaly Detection

Project Overview & Abstract

Frame card fraud as anomaly detection by fitting models primarily on legitimate transactions, then compare probability density scores with reconstruction error on a held out mixed set.

The 'Hybrid' Logic: Gaussian Density Estimation to Learned Nonlinear Reconstruction

AML baseline: Fit a Gaussian Mixture Model on legitimate transactions, tune component count with AIC and BIC, and use low log likelihood as the anomaly score.

Identify the gap: A Gaussian mixture assumes the legitimate transaction manifold has an approximately Gaussian mixture shape and can become unstable in high dimensions.

DL model: Train an undercomplete or denoising autoencoder only on legitimate training transactions; use reconstruction error as the anomaly score.

Fair comparison: Use the same legitimate only training set and untouched mixed test set. Report PR AUC, recall at fixed false positive budgets and precision at a review capacity; do not rely on accuracy.

Suggested Pre-reads

Dempster et al. (1977). Maximum Likelihood from Incomplete Data via the EM Algorithm. Sakurada and Yairi (2014). Anomaly Detection Using Autoencoders.

Datasets & Stack

Recommended starter dataset: Credit Card Fraud Detection dataset from Kaggle and the ULB Machine Learning Group.

Dataset Link: Credit Card Fraud Detection - ULB / Kaggle

Suggested stack: Python, pandas, scikit learn, PyTorch or TensorFlow, matplotlib and calibration or thresholding utilities.

Feature Engineering & Setup Tips

Split chronologically if time is available, fit scaling and GMM only on legitimate training rows, select thresholds on validation data, keep the fraud label hidden during unsupervised training, and report base rates.

Expected Deliverable

A fraud triage notebook with GMM and autoencoder precision recall curves, threshold analysis and documented false positive trade offs.